



EXCALIBUR™

Excalibur

Hardware Design Tutorial

**August 2002
Version 1.5**



101 Innovation Drive
San Jose, CA 95134
(408) 544-7000
<http://www.altera.com>



Copyright © 2002 Altera Corporation. Altera, The Programmable Solutions Company, the stylized Altera logo, specific device designations, and all other words and logos that are identified as trademarks and/or service marks are, unless noted otherwise, the trademarks and service marks of Altera Corporation in the U.S. and other countries. All other product or service names are the property of their respective holders. Altera products are protected under numerous U.S. and foreign patents and pending applications, mask work rights, and copyrights. Altera warrants performance of its semiconductor products to current specifications in accordance with Altera's standard warranty, but reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera Corporation. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services. All rights reserved.



This tutorial provides information about hardware design for the Excalibur™ devices.

Table 1 shows the tutorial revision history.

Table 1. Revision History		
Revision	Date	Description
1.0	Jan 2001	Initial release
1.1	April 2001	Minor amendments; additional full-stripe model tutorial
1.2	June 2001	Minor amendments; new Appendix D
1.3	July 2001	New steps for using Quartus II software mode on PCs and Unix
1.4	April 2002	Minor text amendments. Updated for Quartus II version 2.0.
1.5	August 2002	Added support for Quartus II version 2.1 and GNUPro tools for ARM

How to Find Information

- The Adobe Acrobat Find feature allows you to search the contents of a PDF file. Click on the binoculars icon in the top toolbar to open the Find dialog box.
- Bookmarks serve as an additional table of contents.
- Thumbnail icons, which provide miniature previews of each page, provide a link to the pages.
- Numerous links, shown in green text, allow you to jump to related information.

How to Contact Altera

For the most up-to-date information about Altera® products, go to the Altera world-wide web site at <http://www.altera.com>.

For additional information about Altera products, consult the sources shown in [Table 2](#).

Table 2. How to Contact Altera		
Information Type	USA & Canada	All Other Locations
Technical support	http://www.altera.com/mysupport/	http://www.altera.com/mysupport/
	(800) 800-EPLD (3753) (7:30 a.m. to 5:30 p.m. Pacific Time)	(408) 544-7000 (1) (7:30 a.m. to 5:30 p.m. Pacific Time)
Product literature	http://www.altera.com	http://www.altera.com
Altera literature services	lit_req@altera.com (1)	lit_req@altera.com (1)
Non-technical customer service	(800) 767-3753	(408) 544-7000 (7:30 a.m. to 5:30 p.m. Pacific Time)
FTP site	ftp.altera.com	ftp.altera.com

Note:

(1) You can also contact your local Altera sales office or sales representative.

Typographic Conventions

The *Excalibur Hardware Design Tutorial* uses the typographic conventions shown in [Table 3](#).

Table 3. Conventions

Visual Cue	Meaning
Bold Type with Initial Capital Letters	Command names, dialog box titles, checkbox options, and dialog box options are shown in bold, initial capital letters. Example: Save As dialog box.
bold type	External timing parameters, directory names, project names, disk drive names, filenames, filename extensions, and software utility names are shown in bold type. Examples: f_{MAX} , \qdesigns directory, d: drive, chiptrip.gdf file.
<i>Italic Type with Initial Capital Letters</i>	Document titles are shown in italic type with initial capital letters. Example: <i>AN 75: High-Speed Board Design</i> .
<i>Italic type</i>	Internal timing parameters and variables are shown in italic type. Examples: <i>t_{PIA}</i> , <i>n + 1</i> . Variable names are enclosed in angle brackets (< >) and shown in italic type. Example: <file name>, <project name>.pof file.
Initial Capital Letters	Keyboard keys and menu names are shown with initial capital letters. Examples: Delete key, the Options menu.
“Subheading Title”	References to sections within a document and titles of on-line help topics are shown in quotation marks. Example: “Typographic Conventions.”
Courier type	Signal and port names are shown in lowercase Courier type. Examples: data1, tdi, input. Active-low signals are denoted by suffix n, e.g., resetn. Anything that must be typed exactly as it appears is shown in Courier type. For example: c:\qdesigns\tutorial\chiptrip.gdf. Also, sections of an actual file, such as a Report File, references to parts of files (e.g., the AHDL keyword SUBDESIGN), as well as logic function names (e.g., TRI) are shown in Courier.
1., 2., 3., and a., b., c.,...	Numbered steps are used in a list of items when the sequence of the items is important, such as the steps listed in a procedure.
■	Bullets are used in a list of items when the sequence of the items is not important.
✓	The checkmark indicates a procedure that consists of one step only.
👉	The hand points to information that requires special attention.
↵	The angled arrow indicates you should press the Enter key.
👣	The feet direct you to more information on a particular topic.



Notes:

About this Tutorial	iii
How to Find Information	iii
How to Contact Altera	iv
Typographic Conventions	v
Tutorial	9
Tutorial Files	10
Creating a Design	12
Creating a Quartus II Project	13
Creating an Excalibur MegaFunction	14
Instantiating stripe	20
Verification Using the Bus Functional Model	21
Behavioral Simulation Using the Bus Functional Model	21
Generating .dat Files for Simulation	22
Creating a ModelSim Project	23
Compiling the Design Files	23
Simulating	24
Compilation	26
Compiler Settings	26
Compiling the Design	27
Timing Simulation Using the Bus Functional Model	27
Generating .dat files for Simulation	28
Creating a ModelSim Project	28
Compiling the Design Files	28
Simulating	29
Verification using the Full-Stripe Model	30
RTL Simulation using the Full-Stripe Model	30
Building Software Code Using GNUPro Tools	30
Creating a ModelSim Project	34
Compile the Design Files	35
Compile the Testbench and Model Files	36
Simulating the Design	36
Timing Simulation using the Full-Stripe Model	37
Compiling the Design Files	39
Compiling the Testbench and Model Files	39
Simulating the Design	40
Viewing the Signals	40

Appendix A—
Contents of Mastercommands.dat41

Appendix B—
Contents of Input.dat43

Appendix C—
Contents of Output.dat45

Appendix D—
Simulating VHDL ARM-Based Excalibur Designs47

Using the VHDL Bus Functional Model47

Behavioral Simulation47

Timing Simulation47

Using the VHDL Stripe Model47

Behavioral Simulation48

Timing Simulation48

Appendix E—
Building Software Code Using ADS Tools49

Building the Software Code Using ADS Tools49

viii

Altera Corporation

The *Excalibur Hardware Design Tutorial* guides you through the creation and simulation of a system hardware design for an Excalibur EPXA10 device.



For more information on Excalibur devices, see the [Excalibur Devices Hardware Reference Manual](#).

The design uses the embedded processor core and the AMBA™ high-performance bus (AHB) to write to a memory-mapped slave peripheral in the FPGA portion of the device. The slave peripheral is an arithmetic unit that performs a function based on data received from the embedded processor core. The tutorial shows you how to perform behavioral and timing simulation using a bus functional model and a complete cycle-accurate stripe model. The Excalibur bus transactor, the bus functional model provided in the Quartus® II software, is used along with the ModelSim® software to simulate the design and verify the AHB compatibility of the slave peripheral. The full-stripe model is also used with the ModelSim software to simulate the entire stripe and the slave peripheral logic.

This tutorial involves the following steps:

1. Installing the tutorial
2. Creating a design
3. Behavioral simulation using the bus functional model
4. Compiling the design in the Quartus II software
5. Performing timing simulation using the bus functional model
6. RTL simulation using the full-stripe model
7. Performing timing simulation using the full-stripe model

The instructions assume that:

- Your PC has the following software installed:
 - The Quartus II software, version 2.1
 - The The GNUPro Tools for ARM or Altera ARM Developer Suite (ADS) version 1.1
 - Model Technology™ ModelSim SE software (version 5.6a or above)
- You are familiar with the Quartus II software
- You are familiar with the Excalibur device and its AHB architecture



For more information on the Quartus II software, see the Quartus II Help. For more information on the Excalibur devices, see the *Excalibur Devices Hardware Reference Manual*.

Excalibur designs can be implemented in VHDL, Verilog HDL, AHDL, and graphical format—this walkthrough uses graphical entry and Verilog HDL. For more information on simulating Excalibur designs in VHDL, see *Appendix D— Simulating VHDL ARM-Based Excalibur Designs*.

Tutorial Files

The tutorial files are installed in `C:\altera\excalibur\arm-based\example_designs\arm_tutorial` by default. [Table 1](#) shows the folder structure that is created. If you want to use ADS tools as the embedded software compiler tools, use the files in the `arm_tutorial\ADS` directory. If you want to use GNU tools, use the files in `arm_tutorial\GNU`.

Table 1. Folder Structure

Folder	Description
Arm_tutorial	Contains the Verilog HDL files for the tutorial
\ADS	ADS tutorial files
arm_top.bdf	Quartus II block design file template
p1d_slave.bdf	Slave peripheral block design file
p1d_slave.bsf	Slave peripheral block symbol file
\simulation\modelsim	
bfm_wave.do	BFM waveform command file
edit_mpf.txt	Text to be copied to ModelSim project file
input.dat	Bus transaction command file for BFM
run_rtl_bfm_sim.do	Command file for RTL simulation
run_timing_bfm_sim.do	Command file for timing simulation
slavememory.cfg.dat	FPGA slave memory configuration data
fullmodel_wave.do	Waveform command file
\software	
tutorial_code.s	Assembly code for arm_top design
\GNU	GNU tutorial files
arm_top.bdf	Quartus II block design file template
p1d_slave.bdf	Slave peripheral block design file
p1d_slave.bsf	ModelSim command file that compiles stripe model files.
\simulation\modelsim	
bfm_wave.do	BFM waveform command file
edit_mpf.txt	Text to be copied to ModelSim project file
input.dat	Bus transaction command file for BFM
run_rtl_bfm_sim.do	Command file for RTL simulation
run_timing_bfm_sim.do	Command file for timing simulation
slavememory.cfg.dat	FPGA slave memory configuration data
fullmodel_wave.do	Waveform command file
\software	
tutorial_code.s	Assembly code for arm_top design
rtl	Contains files for RTL simulation with the stripe model
ahb_slave_include.v	Include file that defines the required parameters
ahb_slave_sm.v	The AHB interface state machine that contains all of the logic necessary for the embedded processor core to interface with a slave in the FPGA part of the Excalibur device through the AHB2 FPGA bridge
alu.v	The arithmetic unit that performs the calculations on the two operands
p1d_slave.v	The slave peripheral design

Table 1. Folder Structure	
Folder	Description
regfile.v	Contains the operand, operator, and data registers
\testbench	
arm_top_tb.v	Test bench file

Creating a Design

This section walks you through creating an example of a design—`arm_top.bdf` (see [Figure 1 on page 12](#)).

This design includes the processor core, `arm_processor`, and a memory-mapped slave peripheral, `pld_slave`. `Pld_slave` accepts two operands and an operator that tells it whether to add, subtract, or multiply the operands. The result is written to a location in a register file.

[Figure 2 on page 13](#) shows a block design file representation of the `pld_slave`.

 This section assumes that you are using GNU tools. If you are using ADS tools, replace references to GNU with ADS.

Figure 1. `Arm_top.bdf`

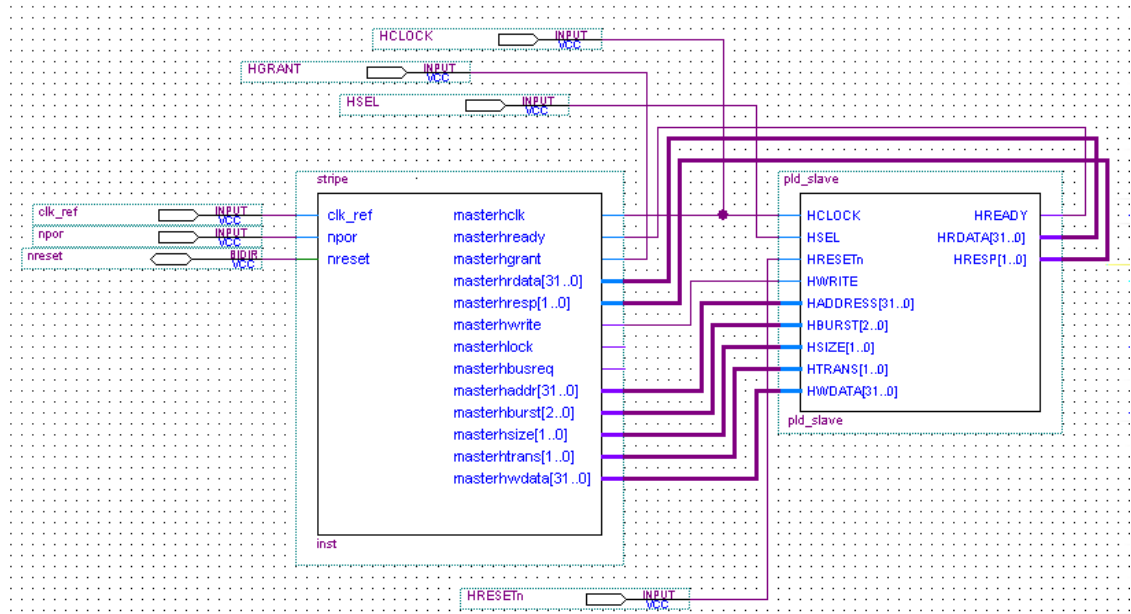
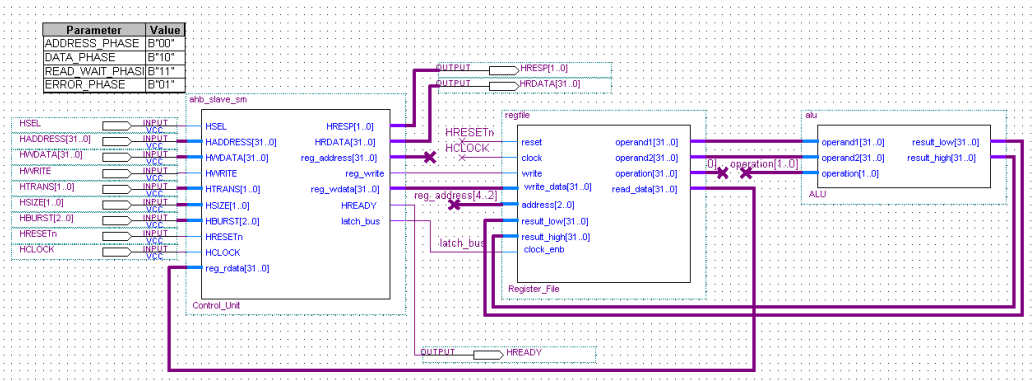


Figure 2. Pld_slave



Creating a design involves the following processes:

- Creating a new Quartus II project and a top-level block design file (.bdf). The .bdf contains arm_processor and pld_slave.
- Synthesizing the design in the Quartus II software.
- Using the Quartus II software to place and route the top-level design see *Compilation* on page 26).

Creating a Quartus II Project

To create a new project, perform the following steps:

1. Open the Quartus II software.
2. Select **New Project Wizard** (File menu) and click **Next** on the Introduction page.
3. Browse to the \arm_tutorial\GNU directory and click **Open**.



This section assumes you are using GNU tools. If you want to use ADS tools to for the embedded software portion of the design, browse to the \arm_tutorial\ADS directory.

4. Type in arm_tutorial as the project name.
5. Type in arm_top as the top-level entity name.
6. Click **Next**.

7. In the Add Files page, browse to `\arm_tutorial\rtl` and select the following files:

`ahb_slave_include.v`

`ahb_slave_sm.v`

`alu.v`

`pld_slave.v`

`reg_file.v`



Using the shift key allows you to select multiple files.

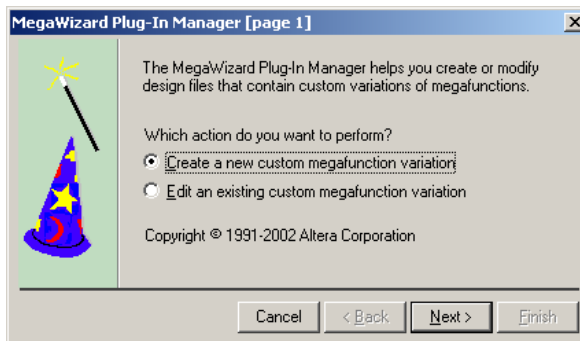
8. Click **Open**.
9. Click **Finish** to create the project.

Creating an Excalibur Megafunction

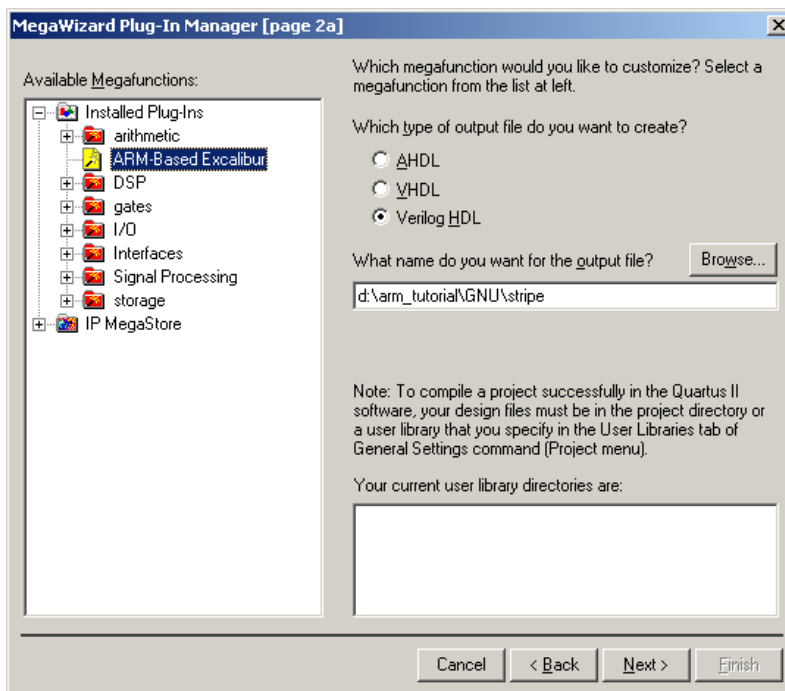
This section creates and instantiates the Excalibur Megafunction into the top-level design file, `arm_top.bdf` using the MegaWizard Plug-In Manager.

Follow these steps to create the symbol:

1. Select **MegaWizard Plug-In Manager** (Tools menu).
2. [Figure 3 on page 15](#) shows the **MegaWizard Plug-In Manager** [page 1] dialog box; select **Create a new custom megafunction variation**.

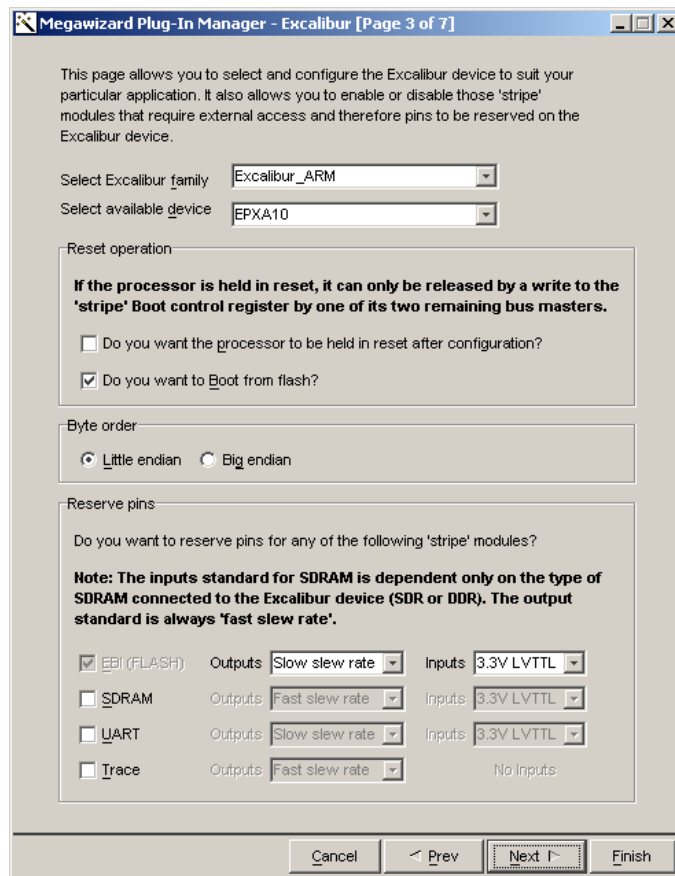
Figure 3. MegaWizard Plug-In Manager: Page 1

3. Click **Next**.
4. **Figure 4** shows the **MegaWizard Plug-In Manager [page 2a]** dialog box; under **Available Megafunctions**, select **ARM-based Excalibur**.

Figure 4. MegaWizard Plug-In Manager: Page 2a

5. Select **Verilog HDL** as the type of output file you want to create
6. Type `\arm_tutorial\GNU\stripe` as the output file name.
7. Click **Next**.
8. **Figure 5** shows the **Megawizard Plug-In Manager – Excalibur [Page 3]** dialog box; select **Excalibur_ARM** as the Excalibur family and **EPXA10** as the available device.

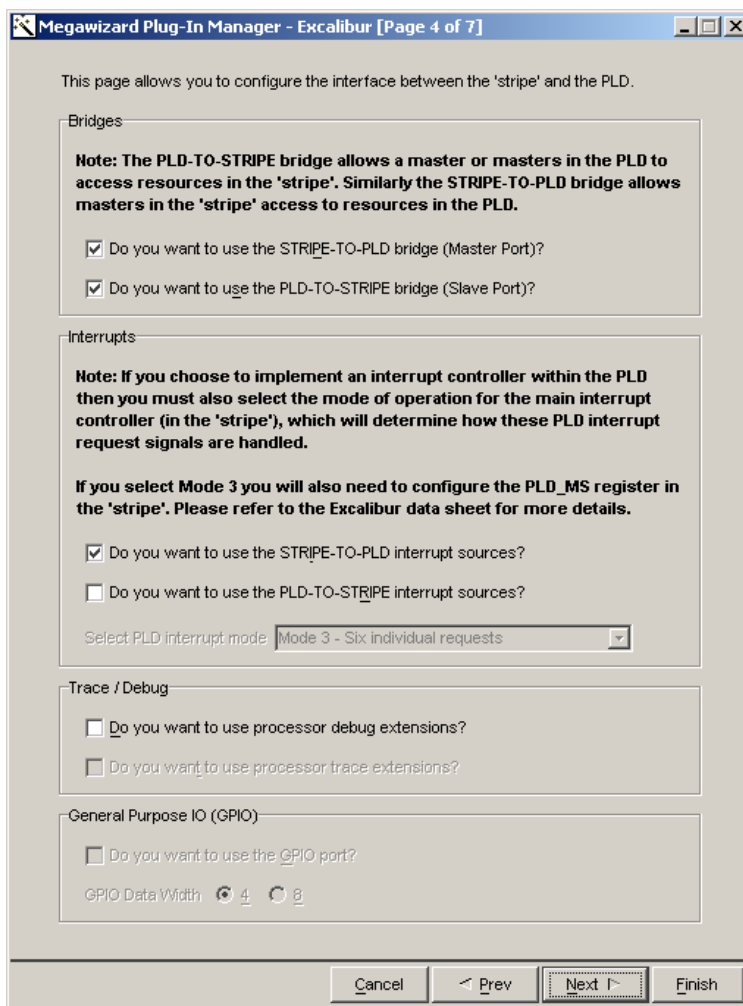
Figure 5. MegaWizard Plug-In Manager: Page 3



9. Turn off both options under **Reset operation**.
10. Select **Little endian** as the byte order.

11. Under the **Reserve pins** heading, ensure that the **EBI (FLASH)**, **SDRAM**, **UART** and **Trace** options are turned off.
12. Click **Next**.
13. Figure 6 shows the **Megawizard Plug-In Manager – Excalibur [Page 4]**; select only the **STRIPE_TO_PLD** bridge (Master Port) option under **Bridges**.

Figure 6. MegaWizard Plug-In Manager: Page 4



14. Turn off both options under **Interrupts**.

15. Ensure that the options under **Trace/Debug** are turned off.



The **General Purpose IO** option is greyed out, because it is only available in EPXA1 and EPXA4 devices.

16. Click **Next**.

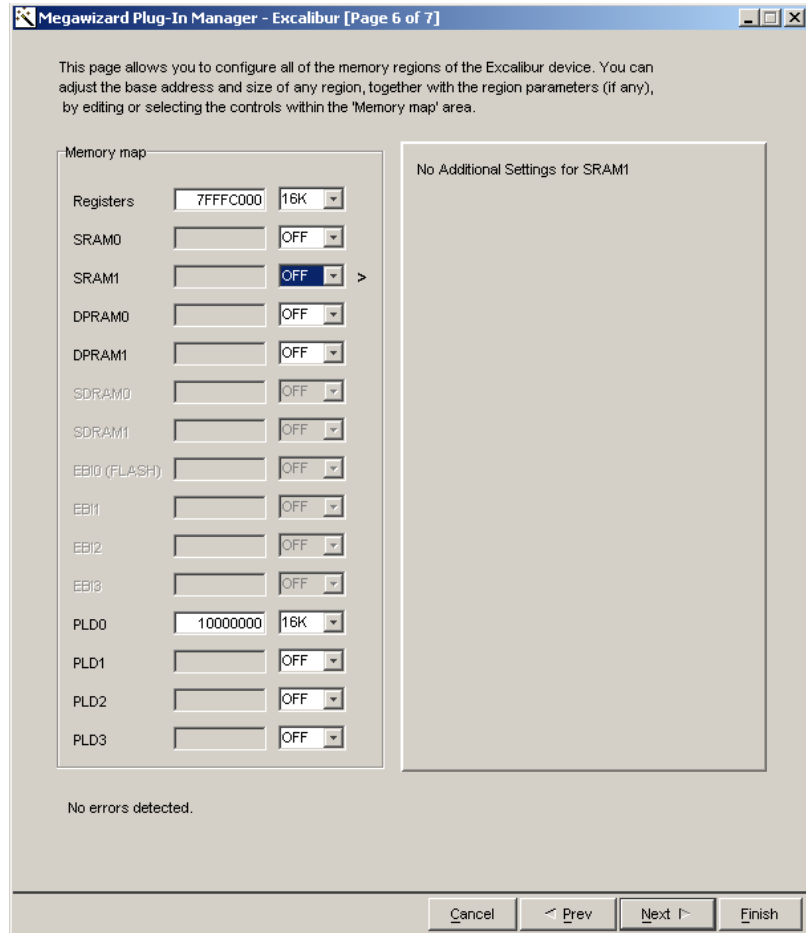
17. **Figure 7** shows the **Megawizard Plug-In Manager – Excalibur [Page 5]**.

Figure 7. MegaWizard Plug-In Manager: Page 5

18. Turn off **Are you using a serial EEPROM configuration device?**

19. Click **Next**.
20. **Figure 8** shows the **Megawizard Plug-In Manager – Excalibur** [Page 6], type **7FFFC000** as the base address of the Registers region and select **16K** as its size.

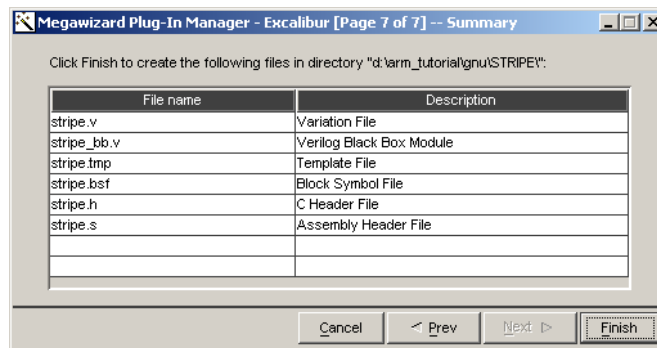
Figure 8. MegaWizard Plug-In Manager: Page 6



21. Leave SRAM0 and SRAM1 to their default mappings, because the tutorial test code is run from on-chip memory.
22. Select **16K** as the size of PLD0 and type **10000000** as the base address.

23. Turn off the **Prefetch** option for PLD0.
24. Set all other region sizes to **off**. The SDRAM and EBI peripherals should be grayed out.
25. Click **Next**.
26. The **Megawizard Plug-In Manager –Excalibur [Page 7]** appears, as shown in [Figure 9](#). Click **Finish** to create the design files.

Figure 9. MegaWizard Plug-In Manager: Page 7



Instantiating Stripe

Instantiate the stripe symbol into the top-level design file, **arm_top.bdf**, by performing the following steps:

 A skeleton of the arm_top design is provided for convenience.

1. Select **Open** (File menu).
2. From the **arm_tutorial\GNU** directory, select **arm_top.bdf**.
3. Click **Open**.
4. Select **Symbol** (Insert menu).
5. In the **Symbol** dialog box, in the **Projects** list, click the + icon to expand the project folder.
6. Select the **stripe** symbol.
7. Click **OK**.

8. An outline of the ARM core symbol is attached to the pointer.
9. Place the symbol by clicking in the center of the block design file so that the stripe pins align with the inputs and outputs as shown in [Figure 1 on page 12](#).
10. Select **Save** (File menu).

Verification Using the Bus Functional Model

This section explains how to perform behavioral simulation using the Excalibur Bus Functional Model (BFM). It shows how to generate the stimulus files for the BFM, how to create a ModelSim project, and what files are needed for the simulation.

Behavioral Simulation Using the Bus Functional Model

You can perform behavioral simulation using the BFM before performing compilation and place-and-route in the Quartus II software. The BFM verifies the functionality of the AHB slaves or masters connected to the Excalibur stripe bridges. You create a top-level Verilog HDL file from your **arm_top.bdf** using the **Create HDL File** utility in the Quartus II software. You can use the resulting **arm_top.v** and lower level Verilog HDL files to simulate the embedded processor-FPGA system in the ModelSim software. The Excalibur bus transactor supplies the **.dat** files required to simulate bus transactions.



For more information on the Excalibur bus functional model, refer to the [Excalibur Bus Functional Model User Guide](#).

To generate the **arm_top.v** file, perform the following steps:

1. If not already open, open **arm_top.bdf** in the Quartus II software.
2. Select **Create HDL Design File for Current File** (Tools menu).
3. Click **Yes**, if you are prompted to save changes to the file.
4. Select **Verilog HDL** in the **Create HDL Design File for Current File** box.
5. Click **OK**.
6. Click **OK** in the box indicating that the HDL file generation was successful.
7. Cut and paste the **arm_top.v** file from the **\arm_tutorial\GNU** folder into the **\arm_tutorial\GNU\simulation\modelsim** folder.



Arm_top.v must not be left in the `\arm_tutorial\tutorial_bfm` folder, because it causes conflicts with **arm_top.bdf** in the Quartus II compilation section of this tutorial.

Generating .dat Files for Simulation

The Excalibur bus transactor is an executable that supports functional simulation of the Excalibur embedded stripe master and slave ports. It accepts **input.dat** as its input and generates a **mastercommands.dat** file. The **mastercommands.dat** file is interpreted by **alteramf.v** or **apex20ke_atoms.v** to simulate read/write instructions issued to the FPGA by the embedded processor via the STRIPE-to-PLD interface. The **alt_exc_stripe_bfm** simulation wrapper file calls the simulation module containing the bus functional model description.

The **input.dat** file (see [Appendix B—Contents of Input.dat](#) for an example) specifies the bus transactions to be simulated. The first command idles the AHB system for four clock cycles. In the `pld_slave`, the register at address 4 is operand 1, the register at address 8 is operand 2, the operation to be done is stored at address 12, and the result is stored at address 16. An operation value of 5 specifies addition, 6 specifies subtraction, and 7 specifies multiplication. The next series of commands writes `0xA` to operand 1, 3 to operand 2, and 5 to the operation register. The expected result from the third command is to read the value 13 (`0DH`), the sum of operands 1 and 2, from the result register. The next series of writes and read exercises the subtraction and multiplication functions of the slave. A burst transaction is also performed, followed by an error-checking test.

To translate the **input.dat** file to a **mastercommands.dat** file, perform the following steps:

1. Open a command prompt window and navigate to the `\arm_tutorial\simulation\modelsim` directory.
2. At the command prompt, type `exc_bus_translate input.dat`.

You see a message in the command window, indicating successful translation. If you get an error indicating that `exc_bus_translate` is not a recognized command, make sure that your system path includes the path to `<QuartusII\bin>`.

3. The **mastercommands.dat** file is now in the `\arm_tutorial\simulation\modelsim` directory.

Creating a ModelSim Project

To create a ModelSim project, perform the following steps:

1. Start ModelSim and select **Run ModelSim** or **Jumpstart**.
2. Click on **Create a Project** in the next window.
3. In the **New Project's Name** window, type `rtl_bfm`.
4. In the **New Project's Home:** window select `\arm_tutorial\GNU\simulation\modelsim`.
5. Click **OK**.
6. Click **Close** when prompted to add files to the project.

The ModelSim software creates a work directory in the `arm_tutorial\GNU\simulation\modelsim` folder.

Compiling the Design Files

To compile the testbench, perform the following steps:

1. Select **Compile** (Compile menu).
2. To add the include file directory, select **Default Options**.
3. Select the **Verilog** tab and click the **Include Dir** button.
4. Browse to `\arm_tutorial\rtl`.
5. Click **Open**.
6. Click **OK**.
7. Browse to `\arm_tutorial\rtl`.
8. Compile the files `alu.v`, `regfile.v`, `ahb_slave_sm.v`, and `pld_slave.v`.



You can select multiple files by holding down the **Ctrl** key on your keyboard.

9. Browse to `\arm_tutorial\GNU` and compile `stripe.v`.
10. Browse to `\GNU\simulation\modelsim` and compile `arm_top.v`.

11. Browse to `<Quartus path>\eda\sim_lib` and compile **altera_mf.v** (the simulation file that contains the bus functional model code).
12. Browse to `<Quartus path>\eda\sim_lib\excalibur\lpm`.
13. Compile **alt_exc_stripe_bfm.v**.
14. Click **Done**.

Simulating

To simulate the design, perform the following steps:

1. In the library window on the left, expand the work library, and double-click on the **arm_top** module to load it.



Several timescale warnings and “too few port connections” warnings, which do not affect the simulation, might appear.

2. Select **Execute Macro** (Tools menu).
3. Select **GNU\simulation\modelsim\bfm_wave.do** and click **OPEN**.

bfm_wave.do sets up the format of the ModelSim waveform window with particular signals in the design. It shows the main AHB signals, inputs, and outputs from the design.



You might see errors regarding unknown options in the waveform depending on your ModelSim version. If the signals are not displayed on the waveform, add the signals (see [Figure 10 on page 25](#)) and re-execute **bfm_wave.do**.

4. Select **Execute Macro** (Tools menu).
5. Select **run_rtl_bfm_sim.do** and click **OPEN**.



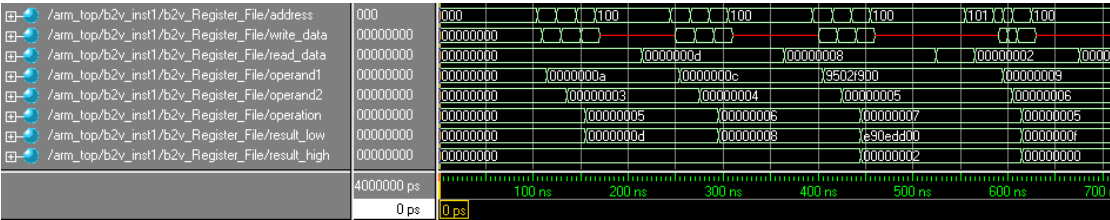
Several timescale warnings and “too few port connections” warnings, which do not affect the simulation, might appear.

run_rtl_bfm_sim.do contains the following commands that provide stimulus to the Modelsim simulation:

```
restart -f
force -drive -repeat 10ns /arm_top/HCLOCK 0 0ns, 1 5ns
force -drive /arm_top/HSEL 0 0ns, 1 10ns
force -drive /arm_top/HRESETn 0 0ns, 1 10ns
force -drive /arm_top/HGRANT 0 0ns, 1 10ns
run 14000ns
```

Open the ModelSim wave window to view the simulation results.
Figure 10 shows a section of the wave window.

Figure 10. Wave Output



The results can also be seen in text format in the **output.dat** file located in the **\arm_tutorial\GNU\simulation\modelsim** folder. The output shows the master commands, addresses, and data values for each transaction specified in the **mastercommands.dat** file. See **Appendix A—Contents of Mastercommands.dat**, which shows the **mastercommands.dat** contents. The simulation confirms that writes and read to the **pld_slave** were performed correctly using the AHB interface.

Compilation

The Quartus II compiler consists of a series of modules that check the design for errors, synthesize the logic, fit the design into an Altera device, and generate output files for simulation, timing analysis, and device programming.

Compiler Settings

To create compiler settings, specify EDA tool settings, and compile the design, perform the following steps:

1. Return to the `arm_tutorial` project in Quartus II.
2. Select **Compile Mode** (Processing menu).
3. Choose **Compiler Settings** (Processing menu). The **General** tab of the **Compiler Settings** dialog box appears automatically.

The **General** tab displays only the default compiler general settings created by the Quartus II software when the project was initially created. These default settings are given the name of the top-level design entity in the project, `arm_top`.

To select the device family and device, perform the following steps:

1. In the **Compiler Settings** dialog box, click the **Chips and Devices** tab.
2. From the **Family** list, select **EXCALIBUR_ARM**.
3. Under **Target Device** tab, select **Specific device selected in Available Devices** list.
4. From the **Available Device** list, select **EPXA10F1020C1**.
5. To accept the defaults for the remaining compiler settings, click **OK**.

Before compiling the design, you must specify EDA tool settings in the Quartus II software:

1. Choose **EDA Tool Settings** (Project menu). The **EDA Tool Settings** dialog box appears.
2. Select **Simulation tool** from the EDA tool type choices, and select **ModelSim (or ModelSim-Altera)** (Verilog HDL output from the Quartus II software) as the tool name.
3. Click **Settings**.

4. Turn off the option **Output Excalibur Stripe as a single module**.



This option controls how the stripe is instantiated in the Verilog netlist output (**.vo**). Turn it off for BFM simulations; turn it on when performing timing simulation with the full-stripe model.

5. Click **OK** in the **Verilog HDL Output Settings** box.
6. Click **OK**.

Compiling the Design

To compile the `arm_top` design, choose **Start Compilation** (Processing menu).

The compiler begins to compile the `arm_top` design entity, and any subordinate design entities, using the `arm_top` compiler settings. The results of the compilation are updated in the **Compilation Report** window.

The compiler may generate one or more of the warning messages that do not affect the outcome of your design, as shown in [Figure 11](#).

Figure 11. Compiler Warning Messages

- ⚠ Reduced register pld_slave:inst1lwait_state_gen:b2v_instlstate_count[3] with stuck data_in port to stuck value GND
- ⚠ Reduced register pld_slave:inst1lahb_slave_sm:b2v_Control_UnitIHRESP[1]*reg0 with stuck data_in port to stuck value GND
- ⚠ Carry chain that starts with CARRY primitive pld_slave:inst1lalub2v_alulpm_mult:mult_69[multcore:mult_core]mpar_add:paddermpar_add:sub_par_addlmpar_add:sub_par_addlmpar_add:s
- ⚠ Data in Simulator initialization files holds embedded processor core in reset because Hexadecimal (Intel-Format) File(s) do not contain initialization data for entry point 0x0 or entry point is in
- ⚠ Data in passive programming files holds embedded processor core in reset because Hexadecimal (Intel-Format) Files do not contain initialization data for entry point 0x0
- ⚠ Data in passive programming files holds embedded processor core in reset because Hexadecimal (Intel-Format) Files do not contain initialization data for entry point 0x0
- ⊞ ⚠ Found pins functioning as undefined clocks and/or memory enables

Timing Simulation Using the Bus Functional Model

Compiling `arm_top` in the Quartus II software generates the files you need to simulate the design in ModelSim. Three files, **`arm_top.vo`**, **`arm_top_v.sdo`**, and **`arm_top_modelsim.xrf`**, are created in the `\arm_tutorial\GNU` folder. You can use these files, and the Excalibur bus functional model, to simulate the Excalibur system in ModelSim.



Because the stripe model is not being used in this section, simulation of `arm_processor` is not performed; functional simulation of the stripe master and slave ports is performed using the bus functional model. The second half of the tutorial uses the full-stripe model.

Generating .dat files for Simulation

You will use the same **mastercommands.dat** file used in the previous behavioral simulation section.



For further information on the **input.dat** file and the **mastercommands.dat** file see [“Generating .dat Files for Simulation” on page 22](#)

Creating a ModelSim Project

To create a ModelSim project, perform the following steps:



If ModelSim is open, select **End Simulation** (Simulate menu) and select **Yes**, which exits any previous project simulations. Select **New** and **New Project** (File menu) and continue with step 2 below.

1. Start ModelSim and choose **Run ModelSim** from the available options.
2. Click on **Create a Project** in the next window or select **New>Project** (File Menu).
3. In the **New Project's Name** window, type `timing_bfm`.
4. In the **New Project's Home** window select `\altera\arm_tutorial\GNU\simulation\modelsim`.
5. Click **OK**. If you are prompted to unload the loaded project, click **OK**.
6. Click **Close** when prompted to add files to the project.
7. Click **Done** on the **ModelSim Welcome** window.

ModelSim creates a work directory in the `\simulation\modelsim` directory.

Compiling the Design Files

To compile the test bench, perform the following steps:

1. Select **Compile** (Compile menu).
2. In the **Files of type** box select **All files(*.*)**.

3. Ensure you are in the `\arm_tutorial\GNU\simulation\modelsim` directory, select **arm_top.vo** and click **Compile**.
4. In the `\<Quartus path>\eda\sim_lib` folder, select **apex20ke_atoms.v** and click **Compile**.
5. Click **Done**.

Simulating

To simulate the design, perform the following steps:

1. In the library window on the left, expand the work library, and double-click on the **arm_top** module to load it.
2. Select **Execute Macro** (Macro menu).
3. Select **run_timing_bfm_sim.do** and click **OK**.



It can take up to 30 minutes for simulation to complete, if you are using the ModelSim-Altera simulation tool.

run_timing_bfm_sim.do contains the following commands that provide stimulus to the ModelSim simulation:

```
restart -f
force -drive -repeat 42ns /arm_top/HCLOCK 0 0ns, 1 21ns
force -drive /arm_top/HSEL 0 0ns, 1 10ns
force -drive /arm_top/HRESETn 0 0ns, 1 10ns
force -drive /arm_top/HGRANT 0 0ns, 1 10ns
run 14000ns
```



The simulation runs at 24 MHz corresponding to the f_{MAX} of the **arm_top** design. Without the arithmetic logic unit, the design runs at up to 81 MHz. The multiplication operation is the main contributor to the design's critical path.

View the simulation results by opening the **output.dat** file (in the `\arm_tutorial\GNU\simulation\modelsim`) folder. The output shows the master commands, addresses, and data values for each transaction specified in the **mastercommands.dat** file. [Appendix A—Contents of Mastercommands.dat](#) details the mastercommands.dat contents; [Appendix C—Contents of Output.dat](#) the expected output.dat simulation results. The simulation confirms that writes and reads to the **pld_slave** were performed correctly using the AHB interface. You can also view the simulation result in the ModelSim waveform window by adding the input pins, output pins, and desired signals to the wave window.

Verification using the Full-Stripe Model

The Excalibur stripe model, a full cycle-accurate model of the Excalibur embedded stripe can be used to verify the functionality of all elements in the stripe except the configuration logic. Software code can be co-simulated with the stripe model. To perform the simulation of software running on the arm_top hardware system, you can use a memory initialization file to model the contents of on-chip SRAM. The simulation shows the execution of instructions, the functionality of the stripe, and the functionality of the FPGA logic.

A testbench file, **arm_top_tb.v**, is provided, to be used for running a simulation of the software with the hardware.

RTL Simulation using the Full-Stripe Model

In this section, you perform RTL simulation of the design using the **arm_top_tb.v** testbench. This instantiates the arm_top module and drives the clk_ref and HCLK clock signals. The testbench also drives the reset signals, HRESETn and npor.

You will simulate the example software, **tutorial_code.s**, running on the hardware design, which first writes to the internal stripe registers to initialize the memory map. It then writes to the pld_slave peripheral registers, operand1, operand2, and operation. The value 0x00 is written to operand1 and 0x03 to operand2. The value 0x06 is written to the operation register to specify subtraction. Subsequently, 0x05 and 0x07 are written to the operation register to specify addition and multiplication, respectively.

To simulate the embedded processor running **tutorial_code.s**, you must first compile it. If you are using ADS tools, refer to [Appendix E— Building Software Code Using ADS Tools](#). The remainder of this section explains how to use the GNUPro tools for ARM.

The compiled code needs to be converted into a memory initialization format, which Quartus II does automatically during compilation. For RTL simulation before the design is compiled, you can use the **makeprogfile** utility to generate the memory initialization files. The command for this is given below in the Software Mode post-build command line.



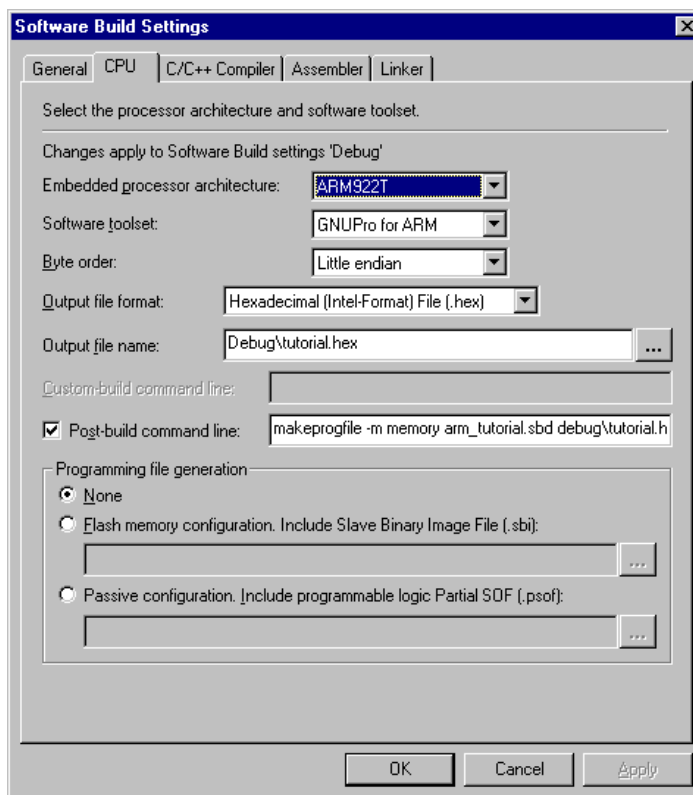
For details on **makeprogfile**, type `makeprogfile -h` at a command prompt.

Building Software Code Using GNUPro Tools

Perform the following steps to compile **tutorial.s** using the GNUPro for ARM tools in Software Mode. (If you are using ADS tools, refer to [Appendix E— Building Software Code Using ADS Tools](#). In this section, you will also convert the software files to memory initialization files to be used in the ModelSim simulation.

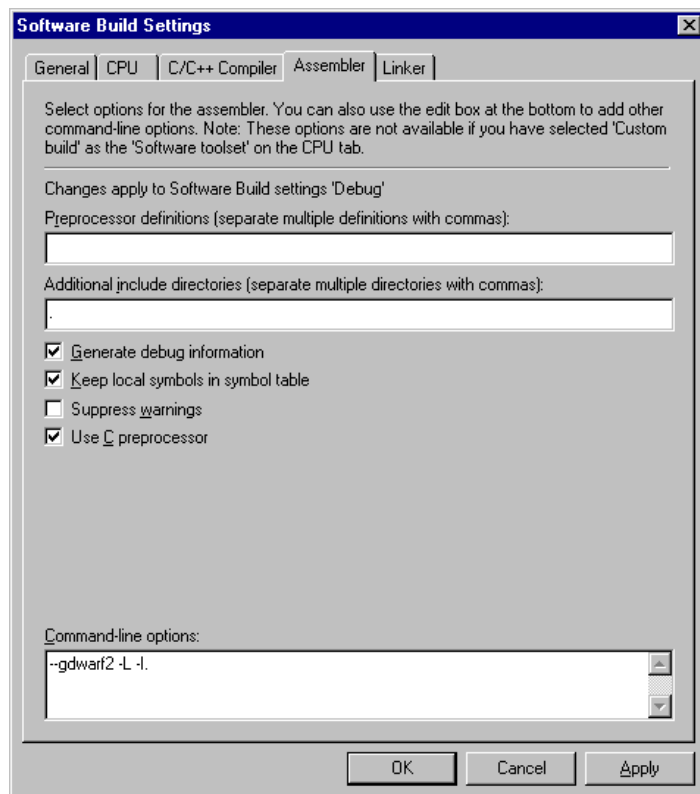
1. In the Quartus II software, choose **Software Mode (Processing menu)**.
2. Go to **Project>General Settings>Toolset Directories** and set the GNUPro for ARM directory to *<GNU cygwin installation directory>/bin*. Click **OK**.
3. Click on the **Add Files** tab and browse to `\arm_tutorial\GNU\software`.
4. Select **tutorial.s** and click **Open**.
5. In the General Settings window, click **OK**.
6. Select **Software Build Settings (Processing menu)**.
7. Click on the **CPU** tab.

Figure 12. Software Build Settings: CPU Tab



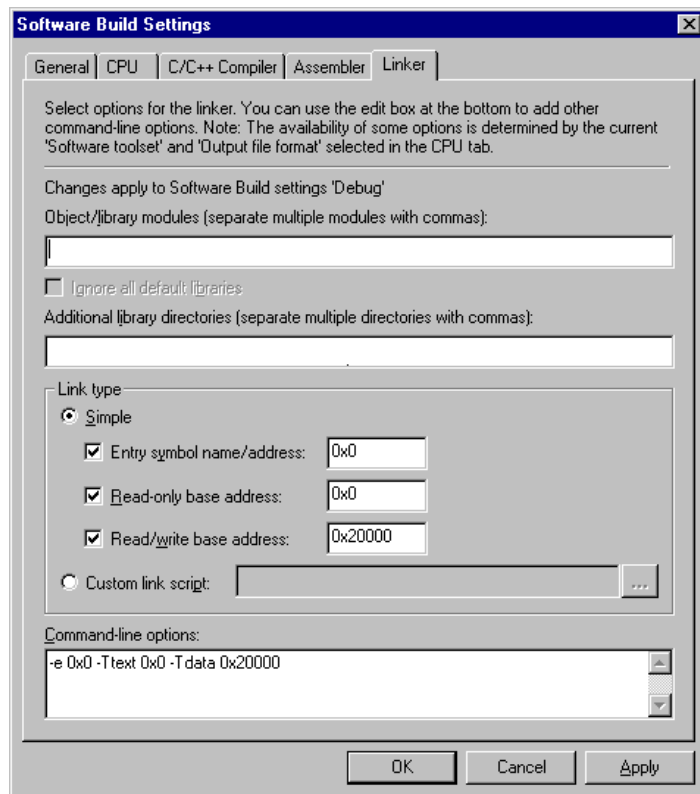
8. Specify the following:
 - **Processor architecture:** ARM922T
 - **Software toolset:** GNUPro for ARM
 - **Byte order:** Little endian
 - **Output file format:** Hexadecimal (Intel-Format) File (.hex)
 - **Output file name:** Debug\tutorial.hex
(When compiling your C code, Quartus creates a **Debug** directory to store the .hex file)
 - Turn on **Post-build command line** and type:
makeprogfile -m memory arm_tutorial.sbd
debug\tutorial.hex
 - **Program file generation:** None
9. Click on the **Assembler** tab.

Figure 13. Software Build Settings: Assembler Tab



10. Specify the following:
 - Additional include libraries: type . to specify the current directory
 - Turn on **Generate debug information**
 - Turn on **Keep local symbols in symbol table**
 - Turn on **Use C preprocessor**
11. Click on the **Linker** tab.

Figure 14. Software Build Settings: Linker Tab



12. Specify the following:
 - **Link type: Simple**
 - Turn on **Entry symbol name/address** and type 0x0
 - Turn on **Read-only base address** and type 0x0
 - Turn on **Read/Write base address** and type 0x20000

13. Click **OK**.
14. Go to **Processing>Start>Software Build** or click  .
15. You must copy the memory initialization files that were generated by the software build into the simulation directory, so that the ModelSim software can read them. Copy the following files to `\GNU\simulation\modelsim`:

```
memory.regs  
memory.sram0  
memory.sram1  
memory.dpram0  
memory.dpram1
```

Creating a ModelSim Project

If your previous ModelSim session is still open, select **End Simulation** (Design menu) and select **YES** to quit simulating the previous project. From the File menu, select **New**, then **New Project**. Continue with step 2 below.

1. Start ModelSim and choose **Run ModelSim** or **Jumpstart** from the options available.
2. Click on **Create a Project** in the next window or select **New>Project** (File Menu).
3. In the **New Project's Name** window, type `rtl_fullmodel`.
4. In the **New Project's Home** window, select `../arm_tutorial/Gnu/simulation/modelsim`.
5. Click **OK**.
6. Click **Close** when prompted to add files to the project.
7. Click **Done** on the ModelSim welcome window.

ModelSim creates a work directory in the `\tutorial_stripemodel\rtl` directory. You must edit the `rtl_fullmodel.mpf` project file by adding the Veriuser variable to the project. This variable provides the ModelSim software with the path to the dynamically loaded objects for Verilog PLI applications and enables the ModelSim software to interface with the Excalibur full-stripe model.

8. Open the **rtl_fullmodel.mpf** file using any text editor.
9. Add the text:

```
Veriuser = $MG_LIB/mti_modelsim_verilog/libmgmm.so
```

under the VSIM section after the Veriuser comments line. (you can cut and paste the text from the **edit_mpf.txt** file in the **\simulation\modelsim** folder).



If a Veriuser line already exists in the project file, append the following text to the existing line:

```
$MG_LIB/mti_modelsim_verilog/libmgmm.so
```

10. Save and close the **rtl_fullmodel.mpf** file.

Compile the Design Files

To compile the design files, perform the following steps:

1. Select **Compile** (Compile menu).
2. To add the include file directory, select **Default Options**.
3. Select the **Verilog** Tab and click the **Add Include Dir** button.
4. If the include directory does not already exist, browse to **\arm_tutorial\rtl**. If it already appears, click **OK** and go to step 7.
5. Click **Open**.
6. Click **OK**.
7. In the **Compile HDL Source** window, browse to the **\arm_tutorial\rtl** directory
8. Compile the following files: **alu.v**, **regfile.v**, **ahb_slave_sm.v**, and **pld_slave.v**. You can select multiple files by holding down the Ctrl key on your keyboard.
9. Browse to **\arm_tutorial\GNU** and compile **stripe.v**.
10. Browse to **\arm_tutorial\GNU\simulation\modelsim** and compile **arm_top.v**.

Compile the Testbench and Model Files

Follow these steps to compile the testbench files and the full-stripe model files.

1. Browse to the `\arm_tutorial\simulation\modelsim` folder.
2. Compile the testbench file, `arm_top_ebi.v`.
3. To compile the model files, browse to `<Quartus installation directory>\eda\sim_lib\excalibur\stripe_model_nt\ModelGen\models\epxa10\r0\mti\modelsim_verilog`, select `alt_exc_stripe.v` and click **Compile**.
4. Click **Done**.

Simulating the Design



For faster simulation times, use the ModelSim PE or SE software rather than the ModelSim-Altera simulation tool, for stripe-model simulation.

To simulate the design, perform the following steps:

1. In the library window on the left, expand the work library and double-click on the `arm_top_tb` module to load it.

You might receive several benign timescale warnings, which do not affect the simulation.

2. Select **Execute Macro** (Tools menu).
3. Select `fullmodel_wave.do` and click **OK**.

`fullmodel_wave.do` sets up the format of the ModelSim waveform window with particular signals in the design. It shows the main AHB signals, inputs, and outputs from the design.

4. Type `run -all` at the ModelSim prompt and click **Enter** to begin the simulation.

The testbench runs for 450 μ s before stopping. Information messages are displayed in the ModelSim window indicating the status of the simulation.

When the simulation completes, you can view the results in the waveform window. The `result_low` and `HRData` bus signals show the results from the ALU operations. You can also view the internal stripe general purpose registers `r0` through `r10` as shown at the bottom of the wave window, shown in [Figure 15](#).

Figure 15. Registers r0 through r10

[illegible]

Timing Simulation using the Full-Stripe Model

To perform timing simulation with the full stripe model, the netlist output from Quartus must contain the appropriate instantiation of the stripe. You must enable the **Output Excalibur Stripe as a Single Module** option and recompile the design using the Quartus II software, as explained below:

1. In the Quartus II software, choose **EDA Tool Settings** (Project menu). The EDA Tool Settings dialog box appears.
2. Choose **Simulation tool** from the EDA tool type drop-down list, and choose **ModelSim** (or ModelSim-Altera) (Verilog HDL output from the Quartus II software) as the tool name.
3. Click **Settings**.
4. Turn on **Output Excalibur Stripe as a single module**.
5. Click **OK** in the Verilog HDL Output Settings box.
6. Click **OK**.

Recompiling the Design

To recompile the design, select **Compile Mode** from the Processing Menu and choose **Start Compilation** (Processing menu).

Create a ModelSim Project



It is recommended that you use the full version of ModelSim rather than the Altera OEM version in this section. The full version gives a faster simulation time.

If ModelSim is still open, select **End Simulation** (Design menu) and select **YES**, which exits any previous project simulations; choose New>Project (File menu) and continue from step 4 below. If not, perform the following steps.

1. Start ModelSim and choose **Run ModelSim** or **Jumpstart**.
2. Click on **Create a Project** in the next window.
3. In the **New Project's Name:** window, type `timing`.
4. In the **New Project's Home:** window select `\arm_tutorial\GNU\simulation\modelsim`.
5. Click **OK**.
6. Select **No** when prompted for a new HDL source file.

The ModelSim software creates a work directory in the **GNU\simulation\modelsim** directory.

You must edit the **timing.mpf** project file by adding the Veriuser variable to the project. This provides ModelSim with the path to the dynamically-loaded objects for Verilog PLI applications. This enables ModelSim to interface with the Excalibur full-stripe model.

7. Open the **timing.mpf** file using any text editor.

8. Add the text, `Veriuser = $MG_LIB\mti_modelsim_verilog/libmgmm.so` under the VSIM section. (you can cut and paste the text from the **edit_mpf.txt** file in the **\modelsim** folder.

If a Veriuser line already exists in the project file, append the following text to it:

```
$MG_LIB\mti_modelsim_verilog/libmgmm.so
```

9. Save and close the **timing.mpf** file.

Compiling the Design Files

Compile the design files, following the steps below:

1. Select **Compile** (Compile menu).
2. Browse to the directory **\arm_tutorial\GNU\simulation\modelsim**.
3. In the **Files of type** box select **All files (*.*)**.
4. Select **arm_top.vo** and click **Compile**.

Compiling the Testbench and Model Files

Use the following steps to compile the testbench files, the full stripe model files, and the **apex20ke_atoms** file.

1. Browse to the folder **\arm_tutorial\GNU\simulation\modelsim**.
2. Compile **arm_top_tb.v**.
3. In the **\<path to Quartus>\eda\sim_lib** directory, select **apex20ke_atoms.v** and click **Compile**.
4. To compile the model wrapper file for timing simulation, browse to **<Quartus II installation directory>\eda\sim_lib\excalibur\stripe_model_nt\ModelGen\models\epxa10\r0\mti_modelsim_verilog**, select **apex20ke_stripe.v** and click **Compile**.
5. Click **Done**.

Simulating the Design

To simulate the design, in the library window on the left, expand the work library and double-click on the arm_top_tb module to load it.

Viewing the Signals

You can view signals in the design by opening the waveform window and viewing the desired signals.

1. Select **Wave** (View menu).
2. Select **Structure** (View menu).
3. Select **Signal** (View menu)
4. Select the structure to which you wish to add signals.
5. In the signals window, select the desired signals.
6. Drag and drop the signals into the waveform window.
7. Type `run -all` at the ModelSim prompt to begin simulation.

The testbench runs for 450 μ s before stopping. Comments indicating the simulation status are displayed in the ModelSim window. The results should match those seen in the RTL simulation section.



2

Appendix A

Altera Corporation

```
// + | + | + * + | | |
// + | + | + * + | | | Line 15, write(12, 32, 1, 6);
000010000000c1000000060020020000//7: write 32-bits NONSEQ SINGLE DATA=6h=6 ADDR=12
// + | + | + * + | | |
// + | + | + * + | | | Line 16, read(16, 32, 1);
00001000000100000000000020020000//8: read 32-bits NONSEQ SINGLE ADDR=16
// + | + | + * + | | |
// + | + | + * + | | | Line 19, write(4, 32, 1, -1794967296);
000010000000419502f9000020020000//9: write 32-bits NONSEQ SINGLE DATA=9502f900h=-1794967296 ADDR=4
// + | + | + * + | | |
// + | + | + * + | | | Line 20, write(8, 32, 1, 5);
00001000000081000000050020020000//10: write 32-bits NONSEQ SINGLE DATA=5h=5 ADDR=8
// + | + | + * + | | |
// + | + | + * + | | | Line 21, write(12, 32, 1, 7);
000010000000c1000000070020020000//11: write 32-bits NONSEQ SINGLE DATA=7h=7 ADDR=12
// + | + | + * + | | |
// + | + | + * + | | | Line 22, read(16, 32, 1);
00001000000100000000000020020000//12: read 32-bits NONSEQ SINGLE ADDR=16
// + | + | + * + | | |
// + | + | + * + | | | Line 23, read(20, 32, 1);
00001000000140000000000020020000//13: read 32-bits NONSEQ SINGLE ADDR=20
// + | + | + * + | | |
// + | + | + * + | | | Line 26, write(4, 32, 3, 9, 6, 5);
00001000000041000000090020120000//14: write 32-bits NONSEQ INC DATA=9h=9 ADDR=4
00001000000081000000060030120000//15: write 32-bits SEQ INC DATA=6h=6 ADDR=8
000010000000c1000000050030120000//16: write 32-bits SEQ INC DATA=5h=5 ADDR=12
// + | + | + * + | | |
// + | + | + * + | | | Line 27, read(16, 32, 1);
00001000000100000000000020020000//17: read 32-bits NONSEQ SINGLE ADDR=16
// + | + | + * + | | |
// + | + | + * + | | | Line 31, write(16, 32, 1, 10);
000010000001010000000a0020020000//18: write 32-bits NONSEQ SINGLE DATA=ah=10 ADDR=16
// + | + | + * + | | |
// + | + | + * + | | | Line 32, read(16, 32, 1);
00001000000100000000000020020000//19: read 32-bits NONSEQ SINGLE ADDR=16
// + | + | + * + | | |
// + | + | + * + | | | Line 33, read(16, 16, 1);
00001000000100000000000020010000//20: read 16-bits NONSEQ SINGLE ADDR=16
// + | + | + * + | | |
// + | + | + * + | | | Line 34, read(24, 32, 1);
00001000000180000000000020020000//21: read 32-bits NONSEQ SINGLE ADDR=24
// + | + | + * + | | |
// + | + | + * + | | |
```

The contents of **input.dat** are:

```
idle(4); /*allow for the BFM to get past is init issues*/

/* Addition test*/
write (4, 32, 1, 0xA);/* write the 32-bit value 0xA into address 4 */
write (8, 32, 1, 3);/* write the 32-bit value 3 into address 8 */
write (12, 32, 1, 5);/* write the 32-bit value 5 into address 12 to add the operands */

read (16, 32, 1);      /* read the 32-bit value from address 16*/

/* Subtraction Test*/
write (4, 32, 1, 0xC);/* write the 32-bit value 0xC into address 4 */
write (8, 32, 1, 4);/* write the 32-bit value 4 into address 8 */
write (12, 32, 1, 6);/* write the 32-bit value 6 into address 12 to subtract the operands */
read (16, 32, 1);      /* read the 32-bit value from address 16*/

/*Multiplication Test*/
write (4, 32, 1, 0x9502F900);/* write the 32-bit value 0x9502F900 into address 4 */
write (8, 32, 1, 5);/* write the 32-bit value 5 into address 8 */
write (12, 32, 1, 7);/* write the 32-bit value 7 into address 12 to multiply the operands */
read (16, 32, 1);      /* read the 32-bit value from address 16*/
read (20, 32, 1);/* read result High*/

/*INCR Burst test*/
write (4, 32, 3, 9, 6, 5);/* burst test */
read (16, 32, 1);      /* read the 32-bit value from address 16*/
```

```
/*Error Checking test*/  
  
write (16, 32, 1, 10);/* Used to check the error condition of writing to a read only register */  
read (16, 32, 1);/* read from the same read only address to see if check works*/  
  
read (16, 16, 1);      /* Used to test the error condition where the incorrect size data is used*/  
  
read (24, 32, 1);      /* Used to test error condition where transaction is not in address space*/
```

The contents of **output.dat** are:

MASTER: trans=[	2]	addr=[00000004]	WRITE data=[0000000a]	expected=[0000000a]	WORD OKAY
MASTER: trans=[	3]	addr=[00000008]	WRITE data=[00000003]	expected=[00000003]	WORD OKAY
MASTER: trans=[	4]	addr=[0000000c]	WRITE data=[00000005]	expected=[00000005]	WORD OKAY
MASTER: trans=[	5]	addr=[00000010]	READ data=[0000000d]	expected=[00000000]	WORD OKAY
MASTER: trans=[	6]	addr=[00000004]	WRITE data=[0000000c]	expected=[0000000c]	WORD OKAY
MASTER: trans=[	7]	addr=[00000008]	WRITE data=[00000004]	expected=[00000004]	WORD OKAY
MASTER: trans=[	8]	addr=[0000000c]	WRITE data=[00000006]	expected=[00000006]	WORD OKAY
MASTER: trans=[	9]	addr=[00000010]	READ data=[00000008]	expected=[00000000]	WORD OKAY
MASTER: trans=[	10]	addr=[00000004]	WRITE data=[9502f900]	expected=[9502f900]	WORD OKAY
MASTER: trans=[	11]	addr=[00000008]	WRITE data=[00000005]	expected=[00000005]	WORD OKAY
MASTER: trans=[	12]	addr=[0000000c]	WRITE data=[00000007]	expected=[00000007]	WORD OKAY
MASTER: trans=[	13]	addr=[00000010]	READ data=[e90edd00]	expected=[00000000]	WORD OKAY
MASTER: trans=[	14]	addr=[00000014]	READ data=[00000002]	expected=[00000000]	WORD OKAY
MASTER: trans=[	15]	addr=[00000004]	WRITE data=[00000009]	expected=[00000009]	WORD OKAY
MASTER: trans=[	16]	addr=[00000008]	WRITE data=[00000006]	expected=[00000006]	WORD OKAY
MASTER: trans=[	17]	addr=[0000000c]	WRITE data=[00000005]	expected=[00000005]	WORD OKAY
MASTER: trans=[	18]	addr=[00000010]	READ data=[0000000f]	expected=[00000000]	WORD OKAY
MASTER: trans=[	19]	addr=[00000010]	WRITE data=[0000000a]	expected=[0000000a]	WORD ERROR
MASTER: trans=[	20]	addr=[00000010]	READ data=[0000000f]	expected=[00000000]	WORD OKAY
MASTER: trans=[	21]	addr=[00000010]	READ data=[0000000f]	expected=[00000000]	HALF WORD ERROR
MASTER: trans=[	22]	addr=[00000018]	READ data=[0000000f]	expected=[00000000]	WORD ERROR



Notes:

If you are designing in VHDL, please use the following guidelines when simulating your design with the bus functional model and the full-stripe model.

Using the VHDL Bus Functional Model

Compile the appropriate simulation files first; these are listed below. Then compile your RTL files in hierarchical order, starting from the lowest-level file. For timing simulation, compile the VHDL netlist output file. You must also specify the standard delay format output file (.sdo) generated by the Quartus II software.

Behavioral Simulation

The following simulation files must be compiled in order when performing behavioral simulation with the bus functional model. They are located in the *<Quartus II installation directory>\eda* directory:

- `\sim_lib\altera_mf.vhd`—megafunction file containing VHDL description of bus functional model
- `\sim_lib\altera_mf_components.vhd`—megafunction component declaration file
- `\excalibur\lpm\alt_exc_stripe.vhd`—Excalibur simulation wrapper file
- `\excalibur\lpm\alt_exc_stripe_arch.bfm`—Excalibur architecture declaration file

Timing Simulation

The following files must be compiled in order when performing timing simulation with the bus functional model. They are located in the *<Quartus II installation directory>\eda* directory:

- `\sim_lib\apex20ke_atoms.vhd`—VHDL simulation models for APEX20KE atoms
- `\sim_lib\apex20ke_components.vhd` - component declaration file

Using the VHDL Stripe Model

The SE version of the ModelSim software is required when using the VHDL Stripe Models.

For VHDL simulations, you do not need to add the "Veriuser" line to your ModelSim project.

Compile the appropriate simulation files first; these are listed below. The simulation files are located in the *<Quartus II installation directory>\eda* directory. Then compile your RTL files in hierarchical order, starting from the lowest-level file. For timing simulation, also compile the VHDL netlist output file. You must also specify the standard delay format output file (.sdo) generated by the Quartus II software.

Behavioral Simulation

The following simulation files must be compiled in order when performing behavioral simulation with the stripe model. They are located in the *<Quartus II installation directory>\eda* directory:

- `\sim_lib\altera_mf.vhd`—megafunction file containing VHDL instantiation of stripe
- `\sim_lib\altera_mf_components.vhd`—megafunction component declaration file
- `\sim_lib\excalibur\stripe_model_<platform>\ModelGen\models\epxa<10,4,or 1>\r0\<simulator and language>\alt_exc_stripe.vhd`—Excalibur simulation wrapper file

Timing Simulation

The following files must be compiled in order when performing timing simulation with the stripe model. They are located in the *<Quartus II installation directory>\eda* directory:

- `\sim_lib\apex20ke_atoms.vhd`—VHDL simulation models for APEX20KE
- `\sim_lib\apex20ke_components.vhd`—component declaration file
- `\sim_lib\excalibur\stripe_model_<platform>\ModelGen\models\epxa<10,4,or 1>\r0\<simulator and language>\apex20ke_stripe.vhd`—wrapper file for stripe model

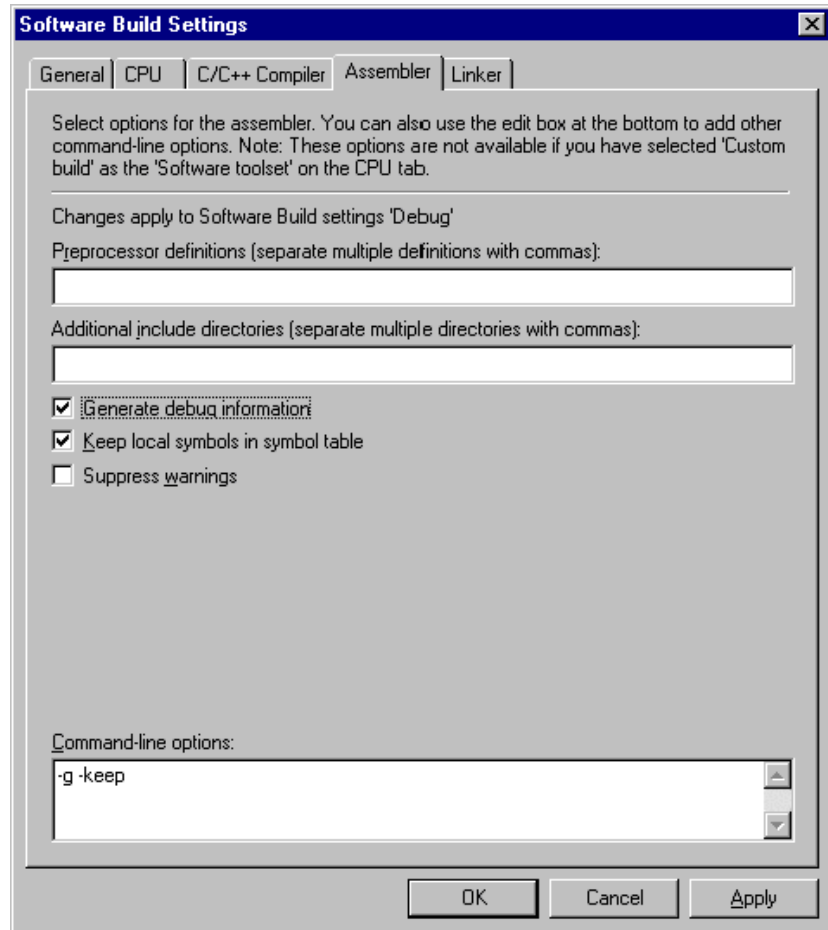
This appendix explains how to use the ADS toolset to build the Excalibur system hardware design.

Building the Software Code Using ADS Tools

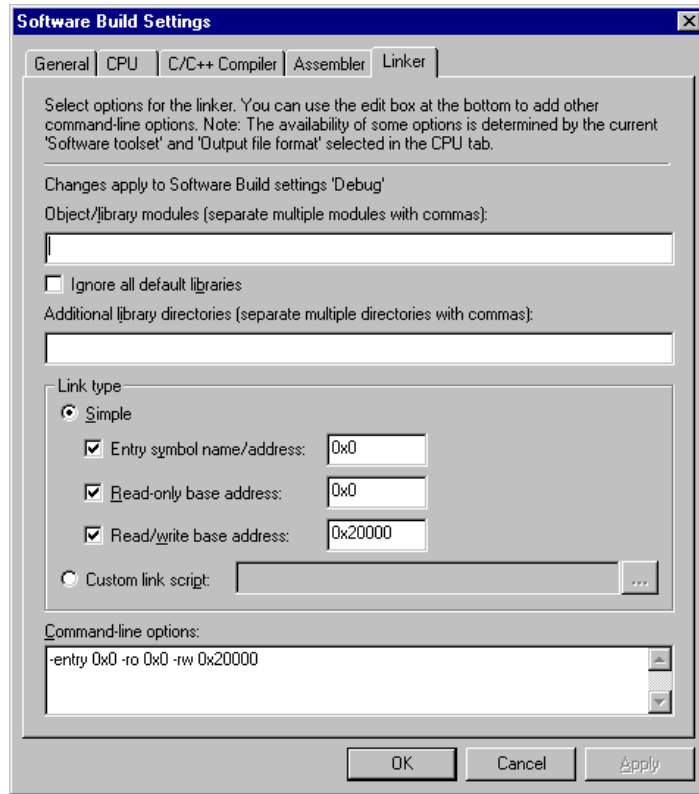
Perform the following steps to compile **tutorial.s** using the ADS tools in Software Mode. In this section, you will also convert the software files to memory initialization files to be used in the ModelSim simulation..

1. Select **Software Mode** (Processing menu).
2. Go to **Project>General Settings >Toolset Directories** and set your ADS Standard Tools directory to *<ADS installation directory>/bin*. Click **OK**.
3. Click the **Add Files** tab and browse to the **\arm_tutorial\ADS\software** folder.
4. Select **tutorial.s** and click **Open**.
5. Click **OK** in the General Settings window.
6. Select **Software Build Settings** (Processing menu).
7. Click on the **CPU** tab.

8. Specify the following:
 - **Processor architecture:** ARM922T
 - **Software toolset:** ADS Standard Tools
 - **Byte order:** Little endian
 - **Output file format:** Hexadecimal (Intel-Format) File (.hex)
 - **Output file name:** type:
Debug\tutorial.hex
(When compiling your code, Quartus creates a **Debug** directory to store the .hex file)
 - Check **Post-build command line** and type:
makeprogfile -m memory arm_tutorial.sbd
debug\tutorial.hex
Program file generation: None
9. Click on the **Assembler** tab.

Figure 1. Software Build Settings: Assembler Tab

10. Specify the following:
 - Turn on **Generate debug information**
 - Turn on **Keep local symbols in symbol table**
11. Click on the **Linker** tab.

Figure 2. Software Build Settings: Linker Tab

12. Specify the following:

- **Link type: Simple**
- Turn on **Entry symbol name/address** and type 0x0
- Turn on **Read-only base address** and type 0x0
- Turn on **Read/Write base address** and type 0x20000

13. Click **OK**.

14. Go to **Processing → Start → Software Mode** or click on .

15. Software mode generates a message indicating that the build was successful.

16. Copy the following files to \ADS\simulation\modelsim:

memory.regs
memory.sram0
memory.sram1
memory.dpram0
memory.dpram1



You must copy the memory initialization files that were generated by the software build into the simulation directory, so that the ModelSim software can read them.

You can now continue with simulation in ModelSim.



Notes: